

Object Oriented Programming

STL (Standard Template Library), Namespaces, Exceptions

PhD. Paul Bustamante

Index

- 1. Review**
- 2. Concept**
- 3. Namespaces**
- 4. `string`**
- 5. `vector`**
- 6. Exceptions**
- 7. Examples**

Review: Class function definition

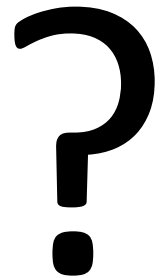
- Class members: variables and functions.
- Functions must be defined **just once**.

Class.h	Class.cpp
<pre>#pragma once #include <iostream> using namespace std; class Class { private: int var1; public: Class(); ~Class(); void Set(int); void Prt(); };</pre>	<pre>#include "Class.h" Class::Class(){} Class::~~Class(){} void Class::Set(int _var1) { var1 = _var1; } void Class::Prt() { cout << var1; }</pre>

Review: Class function definition

- Functions must be defined **just once**.

Class.h	Class.cpp
<pre>#pragma once #include <iostream> using namespace std; class Class { private: int var1; public: Class() {} ~Class() {} void Set(int); void Prt(){}; };</pre>	<pre>#include "Class.h" Class::Class(){} Class::~~Class(){} void Class::Set(int _var1) { var1 = _var1; } void Class::Prt() { cout << var1; }</pre>



Review: Class function definition

- Functions must be defined **just once**.

Class.h	Class.cpp
<pre>#pragma once #include <iostream> using namespace std; class Class { private: int var1; public: Class() {} ~Class() {}; void Set(int); void Prt(){}; };</pre>	<pre>#include "Class.h" Class::Class(){} Class::~Class(){} void Class::Set(int _var1) { var1 = _var1; } void Class::Prt() { cout << var1; }</pre>

Review: Class function definition

Class.h	Class.cpp
<pre>#pragma once #include <iostream> using namespace std; struct my_struct{ double v; char* n; }; class Class{ int var1; public: Class(); ~Class(); void Set(int); void Prt(); Class myfunction(Class c); Class* myfunction2(my_struct a); };</pre>	<pre>#include "Class.h" Class::Class(){} Class::~~Class(){} void Class::Set(int _var1){ var1 = _var1; } void Class::Prt(){ cout << var1; } Class Class::myfunction(Class c){ Class ans; return ans; } Class* Class::myfunction2(my_struct a){ Class *ans; return ans; }</pre>

Output_variable Owner_class :: Function_name (Input_variables)

Review: Constructors

- There must **always** be a **default constructor**.
- Each object is constructed **just once**.

Create a function to set data	// Constructor overloading	// New default/initialization constructor
<pre> class Class { private: int var1; public: Class() {}; // Default constructor void Set(int v1){ var1 = v1;} ~Class() {}; // Default destructor }; </pre>	<pre> class Class { private: int var1; public: Class() {}; // Default constructor Class(int v1){var1 = v1;} ~Class() {}; // Default destructor }; </pre>	<pre> class Class { private: int var1; public: Class() {}; // Default constructor Class(int v1=0){var1 = v1;} ~Class() {}; // Default destructor }; </pre>

Review: Default constructors

- Pointers are copied

```
#pragma once
#include <iostream>
using namespace std;

class Class
{
    int var1;
    int* var2;

    char *var3;
    char var4[30];

public:
    Class(int v1=0, int *v2=(int*)0, char* v3=(char*)"", char* v4= (char*)"")
    {
        var1 = v1;
        var2 = v2;
        var3 = v3;
        strcpy_s(var4, v4);
    }
    ~Class() {}
};
```


Namespace in C++

Namespace provide the space where we can define or declare identifier i.e. variables, methods, classes, etc.

Using **namespace**, you can define the space or context in which identifiers are defined i.e. variable, method, classes. In essence, a **namespace** defines a scope.

Advantage of Namespace to avoid name collision:

1. Example, you might be writing some code that has a function called **xyz()** and there is another library available which is also way of knowing which version of **xyz()** function you are referring to within youhaving same function **xyz()**. Now the compiler has no r code.
2. A **namespace** is designed to overcome this difficulty and is used as additional information to differentiate similar functions, classes, variables etc. with the same name available in different libraries.
3. The best example of namespace scope is the C++ standard library (**std**) where all the classes, methods and templates are declared. Hence while writing a C++ program we usually include the directive using namespace **std**.

A **namespace** definition begins with the keyword namespace followed by the namespace name as follows:

```
namespace namespace_name
{
    int x, y; // code declarations where
              // x and y are declared in
              // namespace_name's scope
}
```

Namespace in C++

```
#include <iostream>
using namespace std;
// first name space
namespace first_space
{
    void func()
    {
        cout << "Inside first_space" << endl;
    }
}
// second name space
namespace second_space
{
    void func()
    {
        cout << "Inside second_space" << endl;
    }
}

using namespace first_space;
int main()
{
    // This calls function from first space.
    func();
    // This calls function from second space
    second_space::func();

    return 0;
}
```

```
#include <iostream>
#include <iomanip>
using namespace std;

namespace food_spc
{
    class product {
        double cost;
        char name[20];
    public:
        product(const char* n, double p) : cost(p) { strcpy_s(name, n); }
        void prt() {
            cout << setw(20) << name << setw(5) << cost << endl;
        }
    };
}

int main()
{
    food_spc::product p1("milk", 1.5);
    p1.prt();

    return 0;
}
```

STL (Standard Template Lib)

Concept of STL's: A set of general-purpose classes and functions, organized into templates, for data storage and processing.

The **Standard Template Library** (STL) is a software library designed for the C++ programming language that influenced many parts of the C++ Standard Library. It provides four components called **algorithms**, **containers**, **functions**, and **iterators**.

There are **4 sequence containers** in C++ STL:

Arrays: The STL array is an implementation of a compile time non-resizable array. It contains various method for common array operations.

Vector: An STL vector can be defined as the dynamic sized array which can be **resized** automatically when new elements are added or removed.

Deque: Deque or Double-Ended Queue is sequence containers with the feature of expansion and contraction on both ends. It means we can add and remove the data to and from both ends.

Lists: List container stores data in non-contiguous memory unlike vectors and only provide sequential access to the stored data. It basically implements the doubly linked list.

C++ String

string: For string manipulation. Example:

Most important functions:

- **size()**: returns string size.
- **operator []**: Access the elements.
- **operator ==, !=, >, <**: compare
- **operator +**: add to string
- **operator >> and <<**.

```
#include <iostream>
#include <string>    //Important for getline
using namespace std;
void main()
{
    string frase1;    //Expression
    cout << "Write a Sentence:";
    getline(cin, frase1);
    cout << "Backwards:"<<endl;
    int len = frase1.size();
    for(int i=len-1;i>=0;i--)    cout << frase1[i];

    string n1,n2;
    cout << "\\nGive 2 names:"; cin >> n1 >> n2;
    if (n1==n2) cout << "Equal names"<<endl;
    else if (n1<n2)
        cout << n1 << " is less than" << n2 << endl;
    else cout << n1 << " is greater than" << n2 << endl;
    string nt = n1 + n2;
    cout << "Together: "<<nt<<" Size="<<nt.size()<< endl;
    cout << "\\nEnd.." << endl;
}
```

C++ Vector

- A vector in C++ is like a resizable [array](#).
- Both vectors and arrays are data structures used to store multiple elements of the same data **type**.
- The difference between an array and a vector, is that the size of an array cannot be modified (you cannot add or remove elements from an array).
- A vector, however, can grow or shrink in size as needed.
- To use a vector, you must include the `<vector>` header file:

// Include the vector library

```
#include <vector>
```

Create a Vector

To create a vector, use the vector keyword, and specify the type of values it should store within angle brackets `<>` and then the name of the vector, like: **vector<type> vectorName.**

Example:

```
// Create a vector called cars that will store strings  
vector<string> cars;
```

C++ Vector Functions

Most important Functions:

- **push_back()**: add elements to the end.
- **size()**: get the size of vector.
- **clear()**: clean the vector.
- **begin()**: posic. To first elem..
- **erase(pos)**: erase some elem.
- **operator []**: access to elements.
- **Iterators**: Allow traversal through the vector, e.g., **begin()** and **end()**.

Additional exercise: Find an item and delete it, using the **erase()** function.

```
#include <iostream>
#include <vector>           //important
using namespace std;
void main()
{
    int Num=0,Max=0, val;
    vector<int> v1;

    while(1){
        cout << "Give val:?"> cin >> val;
        if (val == -1) break;
        v1.push_back(val);
    }
    Num = v1.size();
    cout << "Num= " << Num << endl;
    for(int i=0;i<Num;i++){
        if ( v1[i] > Max) Max = v1[i];
        cout << v1[i] << endl;
    }
    cout << "El Max es:" << Max << endl;
    v1.clear();           //Clean the vector
    cout << "Num= " << v1.size() << endl;
}
```

Example with numbers

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    //vector<int> v1{ 11, 15, 23, 49, 2 }; //create vector with data
    vector<int> numbers;    // Declare a vector of integers
    // Add elements to the vector
    numbers.push_back(10);    numbers.push_back(20);
    numbers.push_back(30);
    // Display the elements of the vector
    cout << "Elements in the vector: ";
    for (int i = 0; i < numbers.size(); i++) {
        cout << numbers[i] << " ";
    }
    // Access elements using iterators
    cout << "\nUsing iterators to display elements : ";
    for (auto it = numbers.begin(); it != numbers.end(); ++it) {
        cout << *it << " ";
    }
    // Modify one element
    numbers[1] = 50; // Change the second element to 50
    cout << "\nAfter modification : ";
    for (int num : numbers) cout << num << " ";
    // Remove the last element
    numbers.pop_back();
    cout << "\nAfter removing the last element : ";
    for (int num : numbers) {
        cout << num << " ";
    }
    numbers.push_back(25);
    numbers.erase(numbers.begin()+1);
    // Size and capacity
    cout << "\nVector size : " << numbers.size() << endl;
    cout << "Vector capacity: " << numbers.capacity() << endl;
    numbers.clear(); // Clear all elements
    cout << "After clearing, vector size: " << numbers.size() << endl;
    return 0;
}
```

Example with strings

```
#include <iostream>
using namespace std;
#include <vector>
void prt(vector<string>& vec)
{
    for (string v : vec) {
        cout << v << endl;
    }
}
int main()
{
    vector<string> cars = { "volvo", "bmw", "ford", "mazda" };
    prt(cars);
    cout << "*****add new car" << endl;
    cars.push_back("opel");
    prt(cars);
    cout << "***** change car" << endl;
    cars[0] = "toyota";
    int len = cars.size();
    for (int i = 0; i < len; i++) {
        cout << i << ":" << cars[i] << endl;
    }
    //erase 1 element
    int ind;
    cout << "Give me the pos to erase (from 0):"; cin >> ind;
    auto pos = cars.begin() + ind;
    cars.erase(pos);
    prt(cars);
    cars.clear(); //erase all elements
    cout << "End!";
}
```


Vector

Example with objects:

```
//file vehiculo.h
#include <iostream>
#include <vector> //important
#include <string>
using namespace std;
class veh
{
    string brand;
    int hp;
public:
    veh(string bd="BYD", int h=100): brand(bd)
        ,hp(h) {}

    void prt() {
        cout << brand << ", " << hp << endl;
    }
};
```

```
vector<veh>::iterator it = lst.begin();
```

```
#include <iostream>
#include <vector>
using namespace std;
#include "vehiculo.h"
int main()
{
    vector<veh> lst;
    veh v1("Toyota", 150);
    lst.push_back(v1);
    lst.push_back(veh("vw", 115));
    lst.push_back(veh("mazda", 200));
    lst.push_back(veh()); //default
    lst.push_back(veh("fiat", 150));

    int len = lst.size();
    for (auto it = lst.begin(); it != lst.end(); it++) {
        veh v = *it;
        v.prt();
    }
    int pos = 0;
    cout << "Erase?"; cin >> pos;
    lst.erase(lst.begin() + pos);

    len = lst.size();
    cout << "----- Other way..\n";
    for (int n = 0; n < len; n++) {
        veh v = lst[n];
        v.prt();
    }
    return 0;
}
```

Vector

Example with objects:

```
//file vehiculo.h
#include <iostream>
#include <vector> //important
#include <string>
using namespace std;

class Vehiculo
{
    string mat;    //matricula
public:
    Vehiculo(string m=""){
        mat = m;
    }
    string GetMat() { return mat; }
    void prt(){
        cout << "Mat: " << mat << endl;
    }
};
```

```
#include "vehiculo.h"
void main()
{
    vector<Vehiculo*> lst;
    while(1){    int opc;
        cout << "\n** Menu **\n1.Add 2.Ver 3.Borrar 4.Salir Opc:?";
        cin >> opc;
        if (opc==1) { string cs;
            cout << "Matricula:?";    cin >> cs;
            Vehiculo *v = new Vehiculo(cs);
            lst.push_back(v);
        } else if (opc==2){ int n = lst.size();
            cout << "Hay " << n << " veh. matriculados:" << endl;
            for(int i=0;i<n;i++){
                Vehiculo *v = lst[i];    v->prt();
            }
        } else if (opc==3) { string mat;
            int n = lst.size();
            cout << "Dar Matricula a borrar:?"; cin >> mat;
            for(int i=0;i<n;i++){
                Vehiculo *v = lst[i];
                if ( mat == v->GetMat() ){
                    lst.erase( lst.begin() + i );
                    break;    //salir del for
                }
            }
        } else { lst.clear();    break; } //fin
    }
}
```

Exceptions

C++ Exceptions

- When executing C++ code, different errors can occur coding errors made by the programmer, errors due to wrong input, or other unforeseeable things.
- When an error occurs, C++ will normally stop and generate an error message. The technical term for this is: C++ will throw an exception (throw an error).

C++ try and catch

- Exception handling in C++ consist of three keywords: try, throw and catch:
- The try statement allows you to define a block of code to be tested for errors while it is being executed.
- The throw keyword throws an exception when a problem is detected, which lets us create a custom error.
- The catch statement allows you to define a block of code to be executed, if an error occurs in the try block.
- The **try** and **catch** keywords come in pairs.

Example

```
try {  
    // Block of code to try  
    throw exception; // Throw an exception when a problem arise  
}  
  
catch () {  
    // Block of code to handle errors  
}
```

Exceptions

Handle Any Type of Exceptions (...)

If you do not know the throw type used in the **try** block, you can use the **"three dots"** syntax (...) inside the **catch** block, which will handle any type of exception:

```
try{  
    int age = 15;  
    if(age >= 18) { cout << "Access granted - you are old enough."; }  
    else {  
        throw 505;  
    }  
}catch(...) {  
    cout << "Access denied - You must be at least 18 years old.\n";  
}
```

Example exceptions

```
#include <iostream>
using namespace std;

int main()
{
    try {
        int age;
        cout << "\nGive me your age:?" << cin >> age;
        if (age >= 18 && age <= 60) {
            cout << "Access granted - you are old enough." << endl;
        }
        else {
            if (age < 18)
                throw 500;
            else
                throw 400;
        }
    }
    catch (int myNum) {
        cout << "Access denied - You must be at least 18 & less than 60.\n";
        cout << "Error number: " << myNum << endl;
    }
    cout << "End!\n";
}
```

Example exceptions

```
#include <iostream>
#include <stdexcept>
using namespace std;
double root(double n) {
    if (n < 0)
        throw runtime_error("Error: less than zero."); //1. end function with exception
    if (n==0)
        throw 1;
    if (n == 1)
        throw 'A';
    if (n == 2)
        throw invalid_argument("Argumento no valido");
    return sqrt(n);
}

void main(){
    double val;
    cout << "Give a number:";
    cin >> val;
    try {
        double r = root(val);
        cout << "root=" << r << endl;
    }
    catch (int &e) {
        cerr << "Error number: " << e << endl;
    }
    catch (exception& e) {
        cout << "ERROR: " << e.what() << endl;
    }
    catch (...) {
        cout << "Generic error" << endl;
    }
    //If you do not know the throw type used in the try block, you can use the "three dots"
    // syntax(...) inside the catch block, which will handle any type of exception :
    cout << "End!\n";
}
```